

Program Drill — Set 5 (hardest)

Three programs with one or two deep defects each, surrounded by correct code that will tempt you

October 2026

How to run it.

Same reading order. 40 minutes per program. Each program has at most two findings that matter; everything else is a positive or a note. Calibration is the skill: write “fine, because...” for the correct lines, then find the one or two that are not.

Program N — Write-behind cache with batched flush

Requirement

1. `set(k, v)` updates the in-memory cache immediately and marks the key dirty.
2. Every `flushMs`, all dirty keys are written to the store in ONE batch call `store.writeMany(entries)`.
3. If the batch write fails, the keys stay dirty and are retried on the next flush; a failure never loses a newer `set` that happened during the write.
4. `close()` performs a final flush and stops the timer; after `close`, `set` throws.
5. `get(k)` returns the cached value or `undefined`.

```
1  type Store = { writeMany(entries: Array<[string, unknown]>): Promise<void> };
2  export class WriteBehind {
3    private cache = new Map<string, unknown>();
4    private dirty = new Set<string>();
5    private timer: ReturnType<typeof setInterval> | null;
6    private closed = false;
7    private flushing = false;
8
9    constructor(private store: Store, flushMs: number) {
10     this.timer = setInterval(() => void this.flush(), flushMs);
11     this.timer.unref?.();
12   }
13
14   get(k: string) { return this.cache.get(k); }
15
16   set(k: string, v: unknown) {
17     if (this.closed) throw new Error("closed");
18     this.cache.set(k, v);
19     this.dirty.add(k);
20   }
21
22   private async flush(): Promise<void> {
23     if (this.flushing || this.dirty.size === 0) return;
24     this.flushing = true;
25     const keys = [...this.dirty];
26     const entries: Array<[string, unknown]> = keys.map((k) => [k, this.cache.get(k)]);
27     try {
28       await this.store.writeMany(entries);
29       for (const k of keys) this.dirty.delete(k);
30     } catch (e) {
31       console.error("flush failed", e);
32     } finally {
33       this.flushing = false;
34     }
35   }
36
37   async close() {
38     this.closed = true;
39     if (this.timer) { clearInterval(this.timer); this.timer = null; }
40     await this.flush();
41   }
42 }
```

Reading sequence

Step	Lines
types	1
state	3–7
signatures	9, 14, 16, 22, 37
bodies	14–20, 37–41
helpers	22–35 (<code>flush</code> , called by the timer and by <code>close</code>)
lifecycle	10–11, 39
clauses	

Notes

Line	Note
10	<code>void this.flush()</code> : <code>flush</code> never rejects (it catches), so <code>void</code> is honest
11	<code>unref</code> so the timer does not hold the process; <code>close</code> still clears it
17–19	<code>set</code> after <code>close</code> throws: clause 4; cache + dirty updated synchronously: clause 1
23–24	overlap guard; no await between: correct
25–26	snapshot of dirty keys and their CURRENT values before the await
28	the batch write; yield point: <code>set(k, v2)</code> can run here for a key in <code>keys</code>
29	on success, deletes every key in the snapshot from <code>dirty</code> , INCLUDING a key that was <code>set</code> again during the write (line 28): that newer value is in <code>cache</code> but no longer dirty; it will never be flushed: clause 3 ("never loses a newer set") violated on the SUCCESS path
30–31	on failure, <code>dirty</code> is untouched: keys retried next flush: clause 3 failure path correct
33	<code>finally</code> resets the guard
40	<code>close</code> awaits a flush; but if a flush is ALREADY in progress (<code>flushing === true</code> from the timer), line 23 returns immediately and <code>close</code> resolves without waiting for it or performing a final flush of keys dirtied during it: clause 4 ("final flush") violated in that window

Clauses

Clause	Status
1 immediate cache + dirty	yes
2 one batch per interval	yes
3 failure keeps dirty; never lose a newer set	failure path yes; SUCCESS path loses a set made during the write (29)
4 close: final flush, stop timer, then set throws	timer yes; set throws yes; final flush skipped if one is in progress (23/40)
5 get	yes

Findings

- Line 29: only un-dirty keys whose value is still the one that was written: compare `this.cache.get(k) === entries[i][1]` before deleting, or track a per-key version. As written, a `set` during `writeMany` is silently dropped.
- Lines 23 and 40: `close` must wait for an in-progress flush and then flush again; keep the in-progress Promise (`this.inflight`) and `await` it in `close` before the final `flush()`.

Positives: overlap guard; snapshot before await; failure path preserves dirtiness; `finally` resets; `unref` plus explicit clear; `void` on a never-rejecting Promise.

Confidence 3 • Request changes.

Both defects are in the two-line window between a snapshot and its commit; the rest is right.

Program O — Paginated sync with checkpoint

Requirement

1. `syncAll()` pulls every page from `api.list(cursor)` and upserts each item via `db.upsert(item)`.
2. After each page is fully upserted, the cursor is saved with `db.saveCursor(cursor)` so a crash resumes from that page.
3. Items within a page may be upserted in parallel (up to 10 at once); pages are sequential.
4. A failing upsert aborts the sync with the error; the cursor must NOT advance past a page with a failure.
5. `syncAll` resolves with the total count of upserted items.

```
1  export async function syncAll(): Promise<number> {
2    let cursor = (await db.loadCursor()) ?? null;
3    let total = 0;
4    while (true) {
5      const page = await api.list(cursor);
6      const results = await Promise.allSettled(
7        page.items.map((item) => limit(() => db.upsert(item)))
8      );
9      const failed = results.find((r) => r.status === "rejected");
10     total += results.length;
11     await db.saveCursor(page.next);
12     if (failed) throw (failed as PromiseRejectedResult).reason;
13     if (!page.next) break;
14     cursor = page.next;
15   }
16   return total;
17 }
```

Reading sequence

Step	Lines
types	none shown; <code>page: { items, next: string null }</code> assumed
state	2–3 (local)
signature	1
body	4–16
helpers	<code>limit</code> (a concurrency limiter, external), <code>db.*</code> , <code>api.list</code>
lifecycle	none
clauses	

Notes

Line	Note
2	resume from saved cursor; <code>?? null</code> for first run
5	sequential pages
6–7	<code>allSettled</code> with a limiter: parallel within the page, bounded: clause 3; <code>allSettled</code> so one failure does not cancel the others (reasonable: finish the page's other items)
9	finds the first rejection
10	counts ALL results, including rejected ones: clause 5 over-counts on failure (minor given line 12 throws anyway, but <code>total</code> is never returned on that path)
11	saves the cursor BEFORE checking for failure: a page with a failed upsert advances the checkpoint; a restart skips the failed items forever: clause 4 violated
12	throws after saving: the order of 11 and 12 is the bug
13	ends on the last page; <code>page.next null</code> saved at line 11 as the checkpoint (fine: next run starts over or treats null as "done", depending on <code>loadCursor</code> semantics; note)
14	advance
5	if <code>api.list</code> throws, the loop exits by exception with the cursor unsaved for the current page: correct (resume re-fetches it)

Clauses

Clause	Status
1 pull all, upsert all	yes
2 checkpoint after each page	yes, but see 4
3 parallel within, sequential across	yes
4 failure aborts; cursor not advanced	abort yes; cursor ADVANCES (11 before 12)
5 total count	over-counts failures; unreachable on failure anyway

Findings

1. Lines 11–12: swap them; save the cursor only after confirming no rejection. As written, a transient upsert failure permanently skips those items.
2. Line 10: count fulfilled only: `results.filter(r => r.status === "fulfilled").length`.
3. Line 13: saving `null` as the final cursor means the next `syncAll` starts from the beginning (full re-sync); decide whether that is intended (note).

Positives: resume from checkpoint; `allSettled` so a page is fully attempted; bounded parallelism via `limit`; sequential pages; failure in `api.list` leaves the cursor at the last good page.

Confidence 2 • Request changes.

One ordering bug that defeats the checkpoint's purpose.

Program P — Request coalescer with TTL and stale-while-revalidate

Requirement

1. `get(key)` returns a cached value if younger than `freshMs`.
2. If older than `freshMs` but younger than `staleMs`, return the stale value immediately AND trigger one background refresh (not one per caller).
3. If older than `staleMs` or absent, wait for a fresh load (coalescing concurrent callers).
4. A failed background refresh keeps the stale value and logs; a failed foreground load rejects the caller(s).
5. Entries are evicted once older than `staleMs` and not refreshed; no unbounded growth.

```
1  type Entry<V> = { value: V; at: number; refreshing: Promise<V> | null };
2  export class SWR<V> {
3    private m = new Map<string, Entry<V>>();
4    constructor(private load: (k: string) => Promise<V>, private freshMs: number, private
staleMs: number,
5                  private now: () => number = Date.now) {}
6
7    async get(key: string): Promise<V> {
8      const e = this.m.get(key);
9      const t = this.now();
10     if (e && t - e.at < this.freshMs) return e.value;
11     if (e && t - e.at < this.staleMs) {
12       if (!e.refreshing) {
13         e.refreshing = this.load(key)
14           .then((v) => { e.value = v; e.at = this.now(); return v; })
15           .catch((err) => { console.error(key, err); return e.value; })
16           .finally(() => { e.refreshing = null; });
17       }
18       return e.value;
19     }
20     if (e?.refreshing) return e.refreshing;
21     const p = this.load(key);
22     const entry: Entry<V> = { value: undefined as V, at: 0, refreshing: p };
23     this.m.set(key, entry);
24     try {
25       const v = await p;
26       entry.value = v; entry.at = this.now(); entry.refreshing = null;
27       return v;
28     } catch (err) {
29       this.m.delete(key);
30       throw err;
31     }
32   }
33 }
```

Reading sequence

Step	Lines
types	1
state	3
signatures	4–5, 7
body	7–32
helpers	<code>load</code> (injected)
lifecycle	none: no sweep
clauses	

Notes

Line	Note
10	fresh: return: clause 1

Line	Note
11–19	stale window: trigger refresh once (guard on <code>e.refreshing</code>), return stale: clause 2; the <code>.then</code> updates in place; <code>.catch</code> keeps the stale value and logs: clause 4 background path; <code>.finally</code> clears the guard: correct; note that <code>.catch</code> RETURNS <code>e.value</code> , so <code>e.refreshing</code> resolves to the stale value rather than rejecting, which matters at line 20
20	expired or absent, but a refresh is in flight: share it: clause 3 coalescing across the stale/expired boundary; because of line 15, a failed refresh resolves this waiter with the STALE value instead of rejecting: clause 4 says a foreground load failure must reject: here a caller who arrived after <code>staleMs</code> gets a stale value silently on failure
21–23	absent: start a load and store a placeholder with <code>refreshing</code> set BEFORE the await: concurrent callers hit line 20: correct coalescing
22	<code>value: undefined</code> as <code>V</code> , <code>at: 0</code> : the placeholder; <code>at: 0</code> means any caller before line 26 completes falls through 10 and 11 (both false since <code>t - 0</code> is huge) to line 20: correct
24–31	foreground await; on failure delete the placeholder and reject: clause 4 foreground path
26	clears <code>refreshing</code> on success
3	no eviction: an expired entry stays in <code>m</code> until someone calls <code>get(key)</code> again (and then only if the reload fails: line 29); a key never requested again stays forever: clause 5 violated
29	on foreground failure the placeholder is deleted, so other waiters (who got <code>p</code> via line 20) reject too (they hold <code>p</code> directly): correct

Clauses

Clause	Status
1 fresh hit	yes
2 stale: return + one background refresh	yes
3 expired/absent: coalesced load	yes
4 background failure keeps stale; foreground failure rejects	background yes; a foreground waiter joining a background refresh gets stale on failure (20 via 15)
5 eviction, no growth	NO: no sweep, entries only removed on foreground failure

Findings

- Line 3: no eviction path for entries older than `staleMs` that are not requested again; add a sweep (interval with `unref`) or evict lazily on every `get` of any key; clause 5.
- Lines 15 and 20: a caller past `staleMs` that joins an in-flight background refresh receives the stale value when that refresh fails, because the `.catch` swallows into a resolved value; either keep two Promises (one raw for foreground joiners, one caught for the background update), or re-check `e.at` after awaiting at line 20 and reject if still expired.

Positives:

single-flight guard on both paths; placeholder entry set before the first await; in-place update of the entry so all references see it; injectable clock; foreground failure deletes the placeholder and rejects every joiner.

Confidence 3 • Request changes.

The algorithm is correct on every path the requirement names except one corner of clause 4, plus the missing eviction.

Scoring yourself

Program	Must-find	Verdict
N	success path un-dirties a key set during the write (29); close skips an in-progress flush (23/40)	3
O	cursor saved before failure check (11–12)	2
P	no eviction (3); stale value returned to an expired joiner on refresh failure (15/20)	3

In all three, more than 80 % of the lines are correct. If your findings list is longer than four items for any program, the extras are not findings; say for each which clause it violates, and if you cannot, strike it.